

**FATEC – FACULDADE DE TECNOLOGIA DE SÃO CAETANO DO SUL
ANTÔNIO RUSSO**

**ISG012 — Atividades 05 e 06: Firewalls de Host e
Netfilter/Nftables**

Efraim de Almeida Lima

**Junho – 2026
SÃO CAETANO - SP**

ÍNDICE

1. Objetivo
2. Atividade ISG012-05 – Teoria: Firewalls de Host: Defesa do Endpoint em SOs
3. Atividade ISG012-06 – Teoria: Netfilter, Iptables e Nftables
4. Conclusão

Objetivo

Esta atividade tem por objetivo aprofundar a compreensão sobre firewalls de host e sua importância na estratégia de defesa do endpoint em sistemas operacionais Windows e Linux (ISG012-05), bem como dominar a arquitetura interna do framework Netfilter, suas ferramentas de configuração (iptables legado e nftables moderno) e a implementação de regras de filtragem stateful e NAT (ISG012-06). Ao concluir, o aluno deverá ser capaz de implementar, auditar e gerenciar firewalls de host em ambas as plataformas, com visão crítica sobre as diferenças de arquitetura e abordagens de automação.

Atividade ISG012-05 – Teoria: Firewalls de Host: Defesa do Endpoint em SOs

Questão 1

Qual a diferença fundamental entre um firewall de host e um firewall de perímetro em termos de escopo de proteção, e por que ambos são necessários em uma arquitetura de defesa em camadas?

Resposta:

O **firewall de perímetro** é posicionado na borda da rede, entre a Internet (ou outra rede não confiável) e a rede interna corporativa. Seu escopo é a proteção do segmento de rede como um todo: controla o tráfego que entra e sai da organização, implementando a política de acesso entre zonas (Internet → DMZ → Rede Interna). Não tem conhecimento dos processos ou aplicações rodando nos endpoints individuais — protege o contorno externo, mas não o interior.

O **firewall de host** é instalado diretamente no sistema operacional de cada endpoint (servidor, workstation, laptop). Opera na camada de aplicação e transporte do próprio dispositivo, filtrando tráfego específico para aquele host — baseado não apenas em endereços IP e portas, mas também em qual executável está realizando a comunicação. Seu escopo é individual: protege aquele endpoint específico, independentemente de onde o tráfego se origina.

Por que ambos são necessários (Defense in Depth):

Proteção contra ameaças internas e movimentação lateral: o firewall de perímetro não protege contra um atacante que já está dentro da rede (funcionário malicioso, dispositivo comprometido, malware que entrou por outro vetor). O firewall de host bloqueia tentativas de acesso lateral entre máquinas internas — mesmo na mesma VLAN.

Proteção em redes não confiáveis: laptops corporativos frequentemente se conectam a redes externas (home office, Wi-Fi de aeroporto). O firewall de host (perfil Public) protege o dispositivo mesmo sem o guarda-chuva do firewall de perímetro corporativo.

Contenção de infecções: se um malware compromete um host interno, o firewall de host pode bloquear sua comunicação com o servidor C2 (Command & Control) via regras outbound, mesmo que o firewall de perímetro não tenha bloqueado o download inicial.

Princípio de defesa em profundidade: a falha de um controle não resulta em comprometimento total — o atacante que passa o perímetro ainda enfrenta o firewall de host de cada dispositivo alvo.

Questão 2

Explique como o Windows Firewall gerencia a aplicação de políticas diferentes através dos perfis Domain, Private e Public, e qual o critério técnico utilizado para determinar qual perfil está ativo em uma determinada interface de rede.

Resposta:

O Windows Firewall implementa três perfis de segurança adaptativos que se ativam automaticamente com base na rede detectada em cada interface:

Perfil Domain:

Ativado quando o Windows detecta que a interface está conectada a uma rede com um controlador de domínio Active Directory autenticado. Aplica as políticas corporativas distribuídas via GPO (Group Policy Object). Tipicamente mais permissivo para serviços empresariais (compartilhamento de arquivos, impressão em rede, WMI, RPC) pois

assume que a rede é gerenciada e confiável.

Perfil Private:

Ativado quando o usuário identifica manualmente a rede como confiável (residencial ou pequeno escritório), ou quando a rede foi previamente marcada como privada. Permite compartilhamento de arquivos e impressoras, descoberta de rede (Network Discovery), mas com restrições moderadas para conexões de entrada.

Perfil Public:

Ativado por padrão para redes desconhecidas ou não identificadas (Wi-Fi de aeroporto, hotel, rede do cliente). Aplica as restrições mais rigorosas: descoberta de rede desabilitada, todas as conexões de entrada bloqueadas exceto as explicitamente permitidas, sem compartilhamento de recursos locais.

Critério técnico para seleção do perfil:

O serviço NLA (Network Location Awareness) do Windows realiza a detecção:

- Verifica se há conectividade com um controlador de domínio AD autenticado via LDAP/Kerberos → **Domain Profile**

- Se não encontrar DC, verifica se a rede foi previamente categorizada pelo usuário como privada → **Private Profile**

- Se nenhuma das condições anteriores for satisfeita → **Public Profile** (padrão seguro)

Cada interface de rede pode ter um perfil diferente simultaneamente. A configuração é feita via `wf.msc` (GUI), GPO (para ambientes corporativos) ou PowerShell:

```
Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled True  
Get-NetFirewallProfile | Select Name, Enabled, DefaultInboundAction
```

Questão 3

Por que as regras outbound (de saída) são tão importantes quanto as regras inbound (de entrada) na estratégia de segurança de um firewall de host, especialmente no contexto de prevenção de ameaças modernas?

Resposta:

Historicamente, firewalls de host focavam quase exclusivamente em regras inbound — bloquear conexões não solicitadas de chegarem ao dispositivo. Contudo, as ameaças modernas exploram ativamente essa lacuna ao operar principalmente através de conexões de saída:

Por que regras outbound são críticas:

Comunicação C2 (Command & Control): malwares modernos (ransomware, RATs, botnets) após infectar um host, estabelecem conexões de saída para servidores C2 controlados pelo atacante, usando portas abertas (80/HTTP, 443/HTTPS) que firewalls tradicionais com foco inbound permitem. Regras outbound que limitam quais processos podem fazer conexões de saída bloqueiam esta fase do ataque.

Exfiltração de dados: malwares e insiders maliciosos transferem dados confidenciais via HTTP/HTTPS, DNS tunneling ou FTP. Regras outbound que restringem conexões saintes apenas a aplicações específicas autorizadas (browsers corporativos, clientes aprovados) dificultam a exfiltração.

Propagação lateral: malwares como worms tentam varrer a rede interna (portas SMB/445, RDP/3389, SSH/22) para se propagar. Regras outbound que bloqueiam essas portas saindo de workstations (que não deveriam iniciar essas conexões) contêm a propagação.

Aplicações não autorizadas: regras outbound por executável (ex.: bloquear cmd.exe, powershell.exe de fazer conexões de rede) detectam e bloqueiam execução de payloads maliciosos dropped por exploits.

No Windows Firewall, as regras outbound podem especificar o caminho do executável:

```
New-NetFirewallRule -DisplayName "Block-PowerShell-Outbound" -Direction Outbound -Program "%SystemRoot%\System32\WindowsPowerShell1.0\powershell.exe" -Action Block
```

No Linux com nftables, as regras outbound são controladas na chain OUTPUT:

```
nft add rule inet firewall output ct state new tcp dport {80, 443} accept
nft add chain inet firewall output { type filter hook output priority 0 \;
policy drop \; }
```

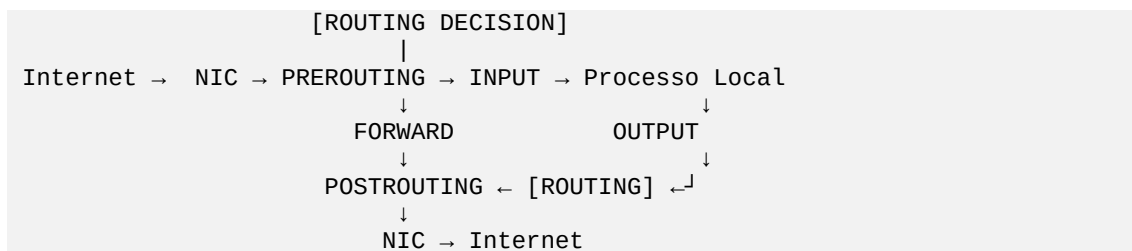
Questão 4

Descreva a arquitetura do Netfilter no Linux, especificando o papel dos hooks (pontos de ancoragem) na pilha de rede e como as tabelas (filter, nat) se relacionam com as chains (INPUT, FORWARD, OUTPUT, PREROUTING, POSTROUTING).

Resposta:

O Netfilter é o framework de filtragem de pacotes integrado ao kernel Linux desde a versão 2.4. Opera no **kernel space** (espaço do kernel), garantindo alta performance e baixa latência. Sua arquitetura é baseada em **hooks** — pontos de interceptação na pilha de rede do kernel onde módulos registram funções de callback para processar pacotes.

Os 5 hooks do Netfilter:



PREROUTING: primeiro ponto de interceptação, antes da decisão de roteamento. Usado para DNAT (alterar destino antes do roteamento decidir para onde vai o pacote).

INPUT: pacotes destinados ao próprio host Linux (após a decisão de roteamento confirmar que o destino é o host local).

FORWARD: pacotes em trânsito pelo host Linux (roteamento entre interfaces) — não destinados ao host em si.

OUTPUT: pacotes gerados pelo próprio host Linux, antes de saírem pela interface.

POSTROUTING: último ponto, após a decisão de roteamento, antes de sair pela interface. Usado para SNAT/masquerade.

Relação tabelas → chains:

Tabela	Propósito	Chains disponíveis
`filter`	Decisão de aceitar/descartar pacotes	INPUT, FORWARD, OUTPUT
`nat`	Tradução de endereços (NAT)	PREROUTING (DNAT), POSTROUTING (SNAT),

		OUTPUT
`mangle`	Modificação de campos dos pacotes (TTL, TOS)	Todos os 5 hooks
`raw`	Controle de connection tracking	PREROUTING, OUTPUT

No nftables, o administrador cria tabelas com nome arbitrário e chains que declaram explicitamente o hook e a prioridade de processamento:

```
nft add table inet meu_fw
nft add chain inet meu_fw input { type filter hook input priority 0 \;
policy drop \; }
```

Questão 5

Quais as vantagens técnicas do nftables em relação ao legado iptables, e como a atomicidade na atualização de regras contribui para a segurança do sistema durante modificações de política?

Resposta:

O nftables foi introduzido no kernel Linux 3.13 (2014) como substituto moderno do iptables, com vantagens técnicas significativas:

1. Sintaxe unificada para IPv4/IPv6/ARP/Bridge:

O iptables requer ferramentas separadas para IPv4 (iptables), IPv6 (ip6tables), ARP (arptables) e bridge (ebtables). Com nftables, a família inet unifica IPv4 e IPv6 em uma única regra, eliminando o risco de políticas assimétricas onde IPv6 fica desprotegido.

2. Compilação para bytecode eficiente:

As regras nftables são compiladas para um bytecode executado por uma máquina virtual no kernel, similar ao eBPF. Isso resulta em performance superior especialmente com conjuntos grandes de regras, pois o kernel não precisa interpretar regras texto uma por uma.

3. Sets e Maps nativos:

```
# Bloquear lista de IPs maliciosos com um único elemento (O(1) lookup vs O(n) iptables)
nft add set inet fw blocked_ips { type ipv4_addr \; }
nft add element inet fw blocked_ips { 1.2.3.4, 5.6.7.8 }
nft add rule inet fw input ip saddr @blocked_ips drop
```


Isso é impossível de forma eficiente no iptables, que criaria uma regra separada para cada IP.

4. Atualizações atômicas de ruleset:

Esta é a vantagem de segurança mais crítica. No iptables, substituir o ruleset completo envolve flush (apagar tudo) e reload (inserir novas regras) — durante esse intervalo o sistema fica sem proteção, uma janela de vulnerabilidade. Com nftables:

```
# Carregar novo ruleset atômicamente — troca instantânea sem janela de vulnerabilidade
nft -f /etc/nftables.conf
```

O nftables aplica todas as novas regras como uma transação atômica — ou o ruleset inteiro é substituído de uma vez, ou nada muda (rollback em caso de erro). Não existe estado intermediário onde algumas regras antigas foram removidas e as novas ainda não foram inseridas.

5. Sintaxe mais legível e menos verbosa:

```
# iptables (legado)
iptables -A INPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate NEW -j ACCEPT
```

```
# nftables (moderno)
nft add rule inet fw input tcp dport {80, 443} ct state new accept
```

Questão 6

Compare a abordagem de gerenciamento de firewalls entre Windows e Linux no contexto de automação e infraestrutura como código, considerando as ferramentas nativas (PowerShell vs bash/nftables) e a integração com sistemas de gerenciamento centralizado.

Resposta:

Windows Firewall — Automação com PowerShell:

O Windows Firewall é gerenciável via módulo `NetSecurity` do PowerShell, que oferece cmdlets completos para CRUD de regras:

```
# Criar regra
New-NetFirewallRule -DisplayName "Allow-HTTPS" -Direction Inbound `
  -Protocol TCP -LocalPort 443 -Action Allow

# Listar regras ativas
```

```
Get-NetFirewallRule | Where-Object {$_.Enabled -eq "True"}

# Exportar configuração como backup
netsh advfirewall export "C:\backup\fw-policy.wfw"
```

Em ambientes corporativos, o Windows Firewall é gerenciado centralizadamente via **Group Policy Objects (GPO)**: políticas de firewall são definidas no Active Directory e propagadas automaticamente para todas as máquinas do domínio em seu próximo login/atualização de política. Ferramentas como SCCM/Intune permitem distribuição e monitoramento em larga escala.

Linux nftables — Automação com bash e IaC:

No Linux, as regras nftables são definidas em arquivos de configuração texto (declarativos), versionáveis no Git e distribuíveis por ferramentas de IaC:

```
# Arquivo /etc/nftables.conf – versionável, auditável
#!/usr/sbin/nft -f
flush ruleset
table inet firewall {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        tcp dport 22 accept
        tcp dport {80, 443} accept
    }
}
```

Para gerenciamento centralizado, integra-se com:

Ansible: módulo `community.general.nftables` para push de configurações

Puppet/Chef: templates de configuração aplicados automaticamente

Terraform: para ambientes cloud com instâncias Linux

Comparação:

Critério	Windows (PowerShell/GPO)	Linux (bash/nftables)
Curva de aprendizado	Moderada (PowerShell)	Moderada (sintaxe nftables)
Centralização nativa	GPO / Active Directory	Ansible, Puppet, Chef
Versionamento	Complexo (registry/GPO)	Git-nativo (arquivos texto)
Auditoria	Event Viewer, PowerShell	journalctl, syslog
IaC (Infra as Code)	Limitado nativamente	Excelente (arquivos texto)
Automação CI/CD	PowerShell DSC	Ansible Playbooks, shell

****Ambientes heterogêneos****

Requer SCCM/Intune

Ansible funciona em qualquer SO

O Linux com nftables é superior em ambientes DevSecOps modernos por tratar configurações de firewall como código versionável, testável e distribuível por pipelines CI/CD — alinhado com práticas de GitOps.

Atividade ISG012-06 – Teoria: Netfilter, Iptables e Nftables

Questão 1 (ISG012-06)

Descreva a arquitetura do Netfilter e explique como iptables e nftables se posicionam em relação a ele.

Resposta:

O Netfilter é o framework de processamento de pacotes no kernel Linux, presente desde a versão 2.4. Funciona como uma infraestrutura de hooks (pontos de interceptação) na pilha de rede do kernel, onde módulos externos registram funções de callback para processar pacotes em momentos específicos de seu ciclo de vida. Opera no **kernel space** com máxima performance.

iptables (legado): é a ferramenta de espaço de usuário que se comunicava com o Netfilter através de chamadas de sistema específicas. Internamente, o iptables utiliza módulos `x_tables` no kernel para armazenar e avaliar as regras. Limitações: sintaxe separada para IPv4/IPv6, performance degradada com regras extensas (avaliação linear), ausência de estruturas de dados avançadas e ausência de atualizações atômicas.

nftables (moderno): introduzido no kernel 3.13 como substituto. Em vez de módulos `x_tables`, utiliza uma **máquina virtual de bytecode no kernel** (similar ao eBPF). As regras são compiladas para bytecode eficiente antes de serem carregadas. O componente de kernel `nf_tables` é mais genérico e flexível. A ferramenta de espaço de usuário `nft` é unificada para todos os protocolos. Atualizações são transações atômicas via netlink.

Ambos acessam os mesmos 5 hooks do Netfilter (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING), mas nftables oferece performance, expressividade e segurança operacional superiores. Distribuições

modernas (Debian 10+, Ubuntu 20.04+, Linux Mint 20+) utilizam nftables por padrão.

Questão 2 (ISG012-06)

Explique a diferença entre as tabelas filter, nat e mangle no Netfilter, indicando em quais chains cada uma opera.

Resposta:

Tabela `filter` — Filtragem de pacotes:

Propósito: decidir se um pacote deve ser aceito (ACCEPT) ou descartado (DROP/REJECT). É a tabela principal de controle de acesso.

Chains disponíveis: INPUT (para o host), FORWARD (em trânsito), OUTPUT (originado no host)

Uso típico: regras de firewall que permitem/bloqueiam serviços específicos, controle de acesso entre segmentos

Tabela `nat` — Network Address Translation:

Propósito: modificar endereços IP e/ou portas nos cabeçalhos dos pacotes. É consultada apenas para o primeiro pacote de uma nova conexão; pacotes subsequentes da mesma conexão seguem a mesma tradução automaticamente via conntrack.

Chains disponíveis: PREROUTING (DNAT — alterar destino), POSTROUTING (SNAT/masquerade — alterar origem), OUTPUT (para pacotes locais roteados)

Uso típico: compartilhamento de Internet (masquerade), publicação de serviços internos (DNAT/port forwarding)

Tabela `mangle` — Modificação de pacotes:

Propósito: modificar campos dos cabeçalhos IP (TTL, ToS/DSCP, marcações de pacotes) sem alterar os endereços de origem/destino. Usado para QoS, marcação de pacotes para Policy-Based Routing e outros fins avançados.

Chains disponíveis: Todos os 5 hooks (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING)

Uso típico: marcar pacotes para roteamento diferenciado (mark), ajustar TTL para VPNs, priorização de tráfego

Tabela `raw` — Controle de connection tracking:

Propósito: decidir quais pacotes não devem ser rastreados pelo connection tracking (NOTRACK), reduzindo overhead para tráfego de alta volumetria conhecido.

Chains disponíveis: PREROUTING, OUTPUT

Questão 3 (ISG012-06)

Demonstre como criar um firewall básico stateful com nftables que permita tráfego SSH (22), HTTP (80) e HTTPS (443) de entrada, bloqueie todo o resto e permita tráfego de saída.

Resposta:

```
#!/usr/sbin/nft -f
# Arquivo: /etc/nftables.conf

# Limpar regras existentes
flush ruleset

table inet firewall {

    # Chain INPUT: tráfego destinado a este host
    chain input {
        type filter hook input priority 0;
        policy drop; # Default: bloquear tudo que não for explicitamente
        permitido

        # Permitir loopback (essencial para funcionamento local)
        iifname "lo" accept

        # Permitir tráfego de conexões já estabelecidas/relacionadas
        (stateful)
        ct state established,related accept

        # Descartar pacotes inválidos (fora de contexto de conexão)
        ct state invalid drop

        # Permitir ICMP (ping) – opcional mas recomendado para
        troubleshooting
        ip protocol icmp accept
        ip6 nexthdr ipv6-icmp accept

        # Permitir SSH
        tcp dport 22 ct state new accept

        # Permitir HTTP e HTTPS
        tcp dport {80, 443} ct state new accept

        # Logar e bloquear o resto
        log prefix "INPUT-DROP: " drop
    }

    # Chain FORWARD: tráfego em trânsito (não aplicável se não for roteador)
    chain forward {
        type filter hook forward priority 0;
        policy drop;
    }
}
```

```
# Chain OUTPUT: tráfego originado neste host
chain output {
    type filter hook output priority 0;
    policy accept; # Permitir todo tráfego de saída (ajustar conforme
necessidade)
}
}
```

Aplicar:

```
nft -f /etc/nftables.conf
systemctl enable nftables
systemctl start nftables
```

Verificar:

```
nft list ruleset
nft list chain inet firewall input
```

Questão 4 (ISG012-06)

Explique o conceito de connection tracking (ct state) no nftables e como ele viabiliza a inspeção stateful.

Resposta:

O connection tracking (módulo `conntrack` do Netfilter, acessado no nftables como `ct`) é o componente que mantém o estado de todas as conexões de rede ativas em uma tabela de estados no kernel. Para cada fluxo de rede identificado pelo quintuplo (IP origem, IP destino, protocolo, porta origem, porta destino), o `conntrack` registra o estado atual:

Estados do conntrack:

- `new`: primeiro pacote de uma nova conexão (ex.: SYN TCP, primeiro datagrama UDP)
- `established`: pacotes pertencentes a uma conexão já vista em ambas as direções (handshake concluído)
- `related`: pacotes associados a uma conexão existente mas em fluxo separado (ex.: mensagem ICMP "port unreachable" referenciando uma conexão UDP, canal de dados FTP iniciado pela sessão de controle)
- `invalid`: pacotes que não correspondem ao estado esperado (ex.: RST sem conexão ativa, ACK sem SYN precedente)
- `untracked`: pacotes marcados para não rastreamento via tabela `raw`

Como viabiliza a inspeção stateful:

```
# Sem ct state (stateless) – necessita de regras bidirecionais
tcp dport 22 accept          # permite entrada na porta 22
tcp sport 22 accept          # precisaria disso para permitir respostas
(perigoso: qualquer tráfego com sport 22)

# Com ct state (stateful) – regra unidirecional + resposta automática
tcp dport 22 ct state new accept      # permite apenas o início de novas
conexões SSH
ct state established,related accept    # respostas a conexões legítimas
passam automaticamente
ct state invalid drop                 # pacotes anômalos são descartados
```

O `ct state established,related accept` é a regra mais importante de um firewall stateful: garante que respostas a conexões iniciadas pelo host (ou permitidas para entrada) passem automaticamente sem necessidade de regras de retorno explícitas — e sem abrir portas genéricas que permitiriam conexões não solicitadas.

Questão 5 (ISG012-06)

Compare iptables e nftables em termos de sintaxe, performance e capacidades de gerenciamento, fornecendo exemplos equivalentes.

Resposta:

Comparação de sintaxe — mesma regra em ambas as ferramentas:

```
# Permitir TCP nas portas 80 e 443 com stateful

# iptables (legado) – requer módulo multiport e verbose
iptables -A INPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate
NEW,ESTABLISHED -j ACCEPT
iptables -A OUTPUT -p tcp -m multiport --sports 80,443 -m conntrack --
ctstate ESTABLISHED -j ACCEPT

# nftables (moderno) – expressivo e conciso
nft add rule inet fw input tcp dport {80, 443} ct state new,established
accept
nft add rule inet fw output tcp sport {80, 443} ct state established accept
```

Bloquear uma lista de IPs:

```
# iptables – uma regra por IP (O(n) linear scan)
iptables -A INPUT -s 192.168.1.100 -j DROP
iptables -A INPUT -s 192.168.1.101 -j DROP
iptables -A INPUT -s 10.0.0.50 -j DROP
# ... 1000 regras = 1000 avaliações por pacote
```

```
# nftables – set com lookup O(1)
nft add set inet fw blacklist { type ipv4_addr \; }
nft add element inet fw blacklist { 192.168.1.100, 192.168.1.101,
10.0.0.50 }
nft add rule inet fw input ip saddr @blacklist drop
# ... 1 regra independentemente do tamanho da lista
```

Tabela de comparação:

Critério	iptables	nftables
IPv4/IPv6 unificado	✗ Ferramentas separadas	✓ Família `inet`
Performance (regras extensas)	Degradação linear	O(1) via sets/maps
Atualizações atômicas	✗ Flush + reload = janela de vulnerabilidade	✓ Transação atômica
Sets/Maps nativos	✗ Apenas via ipset separado	✓ Nativo
Bytecode no kernel	✗	✓
Legibilidade	Verbosa	Concisa
Suporte em distros modernos	Legado (disponível)	Padrão atual

Questão 6 (ISG012-06)

Demonstre como configurar NAT (SNAT e DNAT) com nftables para um cenário de gateway doméstico/corporativo.

Resposta:

Cenário:

Interface externa (eth0): 203.0.113.1 (IP público)
Interface interna (eth1): 192.168.1.1 (gateway)
Rede interna: 192.168.1.0/24
Servidor web interno: 192.168.1.10:80
IP forwarding: ativado (sysctl -w net.ipv4.ip_forward=1)

```
#!/usr/sbin/nft -f

flush ruleset

table inet nat {

    # DNAT – redirecionar tráfego entrante para servidores internos (PREROUTING)
    chain prerouting {
```



```

    type nat hook prerouting priority dstnat;

    # Redirecionar HTTP externo (porta 80) para servidor web interno
    iifname "eth0" tcp dport 80 dnat to 192.168.1.10:80

    # Redirecionar HTTPS externo (porta 443) para servidor web interno
    iifname "eth0" tcp dport 443 dnat to 192.168.1.10:443

    # Redirecionar SSH externo porta 2222 para servidor SSH interno
    porta 22
    iifname "eth0" tcp dport 2222 dnat to 192.168.1.20:22
    }

    # SNAT – mascarar tráfego sainte da rede interna (POSTROUTING)
    chain postrouting {
        type nat hook postrouting priority srcnat;

        # Masquerade: IP de origem privado → IP público da interface eth0
        # Usar masquerade (dinâmico) para ambientes com IP variável
        oifname "eth0" masquerade

        # Alternativa com IP estático (mais eficiente):
        # oifname "eth0" snat to 203.0.113.1
    }
}

table inet filter {

    chain forward {
        type filter hook forward priority 0;
        policy drop;

        # Tráfego estabelecido/relacionado (respostas)
        ct state established,related accept

        # Permitir rede interna → Internet
        iifname "eth1" oifname "eth0" ct state new accept

        # Permitir acesso ao servidor web (após DNAT)
        iifname "eth0" oifname "eth1" ip daddr 192.168.1.10 tcp dport {80,
443} ct state new accept

        # Permitir acesso SSH interno (após DNAT)
        iifname "eth0" oifname "eth1" ip daddr 192.168.1.20 tcp dport 22 ct
state new accept
    }
}

```

Aplicar e verificar:

```

nft -f /etc/nftables.conf
nft list ruleset
# Testar SNAT: de host interno, acessar site externo
# Testar DNAT: de fora, acessar o IP público na porta 80

```

Conclusão

Esta atividade consolidou a última camada da arquitetura de defesa em profundidade estudada na disciplina: a proteção individual do endpoint via firewalls de host. O módulo ISG012-05 demonstrou que a segurança não termina no perímetro — cada dispositivo deve ter sua própria política de firewall adaptativa (perfis Domain/Private/Public no Windows; chains input/output no Linux), incluindo regras outbound que são frequentemente negligenciadas mas críticas para conter ameaças modernas.

O módulo ISG012-06 forneceu a base técnica para implementação eficaz no Linux, evidenciando a superioridade arquitetural do nftables sobre o legado iptables: atualizações atômicas que eliminam janelas de vulnerabilidade, sets nativos com lookups $O(1)$, sintaxe unificada para IPv4/IPv6 e integração nativa com ferramentas de infraestrutura como código. Dominar o Netfilter/nftables é uma competência fundamental para qualquer profissional de segurança que trabalhe com infraestruturas Linux — o sistema operacional mais presente em servidores, roteadores e dispositivos de segurança do ecossistema corporativo atual.